

University of Central Punjab

Faculty of Information Technology

Object Oriented Programming

Fall 2025

Lab 10	
Topic	Classes in C++, access specifiers, objects, methods, constructors, destructors, Inheritance, virtual and overridden members, assigning subclass object to superclass pointers.
Objective	Making students familiarize with the concepts of constructors, destructors, Use inheritance, Polymorphism, overriding members and functions, Observing behavior by assigning objects to superclass pointer.
CLO	Apply object-oriented programming principles to implement real-world problems.

Instructions:

- Indent your code.
- Comment your code.
- Use meaningful variable names.
- Plan your code carefully on a piece of paper before you implement it.
- Name of the program should be same as the task name. i.e. the first program should be Task_1.cpp
- **void main() is not allowed. Use int main()**
- **You are not allowed to use any built-in functions**
- **You are required to follow the naming conventions as follow:**
 - o **Variables:** firstName; (no underscores allowed)
 - o **Function:** getName(); (no underscores allowed)
 - o **ClassName:** BankAccount (no underscores allowed)

Students are required to complete the following tasks in lab timings.

Task 1:

Inheritance (Person + Employee -> Manager)

Base Class: Person

Attributes:

- **name (char *)** – Name of the person.
- **age (int)** – Age of the person.

Functions:

- a) **Constructor:** Implement a parameterized constructor to initialize name and age.
- b) Implement **getter and setter functions** to initialize and retrieve the attributes of the Person class.
- c) **displayPerson()** – Implement a function that prints the name and age of the person.

Base Class: Employee

Attributes:

- **employeeID (int)** – Unique ID for the employee.
- **Department (char *)** – Department name of the employee.

Functions:

- d) **Constructor:** Implement a parameterized constructor to initialize employeeID and department.
- e) Implement **getter and setter functions** to initialize and retrieve the attributes of the Employee class.
- f) **displayEmployee()** – Implement a function that prints the employeeID and department of the employee.

Derived Class: Manager

Attributes:

- **int teamSize** – Number of employees in the manager's team.

Functions:

- g) **Constructor:** Implement a parameterized constructor to initialize teamSize and call the constructors of Person and Employee.
- h) Implement **getter and setter functions** to initialize and retrieve the teamSize attribute.
- i) **displayManagerInfo()** – Implement a function to display all details of the manager (including inherited attributes).

Main Function:

1. Create an object of the Manager class, providing values for Person, Employee, and Manager attributes.
2. Call or retrieve the values of Person and Employee attributes using the Manager class object.
3. Call the display() functions of both base classes using the Manager object to display all attributes.

Task 2:

Virtual Functions and Method Overriding

Base Class: Animal

Attributes:

- **name (char *)** – Name of the animal.

Functions:

- a) **Constructor:** Implement a parameterized constructor to initialize name.
- b) Implement a public **setter function** to set the name attribute.
- c) **display()** – Implement a function that prints the name of the animal.

Virtual Function:

- d) **void makeSound()** – A virtual function that will be overridden in derived classes. The function will print a message “Base class: Animal object”.

Derived Class (From Animal): Dog

Functions:

- a) **Constructor:** Implement a constructor that calls the base class constructor.
- b) Override the **makeSound()** function to print "Derived class: Dog object".

Derived Class (From Animal): Cat

Functions:

- a) **Constructor:** Implement a constructor that calls the base class constructor.
- b) Override the **makeSound()** function to print "Derived class: Cat object".

Main Function:

1. Create an object of base class Animal.
2. Create an object of derived class Cat.
3. Create an object of derived class Dog.
4. Call **display** function for all the objects and observe their outputs.
5. Call **makeSound** function for all the objects and observe their outputs.
6. Now assign subclass object to superclass object, and try to call subclass **makeSound()** function using superclass object and observe the behavior.
7. Now make the **makeSound()** function as virtual function in base class Animal.

8. Make pointer for superclass function Animal, and try to call **makeSound()** function using superclass pointer.
9. Now assign subclass object to superclass pointer, and try to call subclass **makeSound()** function using superclass pointer and observe the output.

Task 3:

Assigning Subclass Objects to Superclass Objects and Pointers

1. Create a Base Class: Teacher

Attributes:

- **protected string name** – Name of the teacher.
- **protected int age** – Age of the teacher.
- **protected string qualification** – Qualification of the teacher.
- **protected int numberOfStudents** – Number of students taught by the teacher.

Functions:

- a) Implement public setter and getter functions for each data member.

2. Create Subclasses: Permanent and Visiting

- b) Leave both subclasses empty for this initial implementation.

3. Implementation in Main Function:

- a) Create a single object of the subclass **Permanent**.
- b) Use **setter functions** to set the values of name, age, qualification, and numberOfStudents.
- c) Print the details of the **Permanent** object with the heading "Permanent Staff".

4. Assigning Subclass Object to Superclass Object:

- a) Assign the **Permanent** object to a **Teacher** type object.
- b) Print the details using getter functions to verify access.

5. Create an Array for Visiting Staff:

- a) Create an array of 3 Visiting objects.
- b) Use setter functions to set values for each Visiting object and print them in a table format with the heading "Visiting Staff".

6. Using Pointers to Access Objects:

- a) Create a pointer of type **Teacher**.
- b) Assign it the address of the **Permanent** object. Use the pointer to access and print the details with the heading "Permanent Staff accessed using Superclass Pointer".
- c) Now, assign the address of the **second Visiting object to the pointer**. Call setter functions to set values for the Visiting object using the pointer.
- d) Print the entire array of **Visiting objects using the pointer** with the heading "Visiting Staff with Object 2 Set using Superclass Pointer".

7. Accessing with Another Pointer:

- a) Create another **pointer of type Teacher** and assign it the address of the first element of the **Visiting objects array**.
- b) Print the whole array using this pointer with the heading "Visiting Staff Accessed Using Pointer".

8. Implementing Overriding:

- a) **Introduce a new data member protected** string '**schoolName**' in the Permanent class to override the name attribute.
- b) Provide a **default constructor** in Permanent to initialize schoolName with a name of your choice.
- c) Implement the **show()** method in the Permanent class to print both the overridden name and schoolName.
- d) Create another **object of Permanent** in the main function.
- e) Call **set functions** to set the superclass data members, then call get functions to print these members.
- f) **Call the show() method** to print the schoolName and analyze the output, noting that set functions initialize the superclass member, not the overridden subclass member.